NFL Combine - RB vs WR Athleticism Project

by Nikhil Young and Kaiya Lee

Project 1

Principal Component Analysis (PCA) is fundamentally a dimensionality reduction algorithm that transforms high-dimensional data into a lower-dimensional space while preserving the most important variance in the dataset. By identifying the principal components—the directions of maximum variance—PCA enables us to reduce computational complexity and mitigate the curse of dimensionality.

Beyond dimensionality reduction, PCA serves multiple practical purposes in data science and machine learning. As a visualization tool, it allows us to project complex, high-dimensional datasets into two or three dimensions, making patterns and clusters visible to the human eye. For noise filtering, PCA can separate the signal from noise by capturing only the components with significant variance while discarding low-variance fluctuations.

In feature extraction and engineering, PCA creates new features that are linear combinations of the original variables, often revealing underlying structure in the data that wasn't immediately apparent. PCA finds applications in data compression, enhancing detections and as a preprocessing step to improve the performance and training speed of machine learning algorithms.

Find a dataset and perform PCA to highlight its key concepts and provide outputs accordingly.

Grading Rubric/Criteria:

- 5 Dataset used - provide source information or dataset < 10 points.
- 2 Clear libraries used - 20 points.
- 15 Perform necessary tasks associated with PCA - Scaling, dimensionality reduction, etc., as needed - 75 points.
- 5 Provide necessary visualizations/graphs - 75 points.
- 5 Provide comments as needed to explain your analysis - 20 points.

What you need to submit:

- 1 Your code as a Jupyter/Colab notebook.
- 2. A PPT with 2-3 slides max. including the introduction slide:
  - Slides to have dataset information - source
  - Explain the concept of PCA and its application to your dataset
  - Explain your analysis and results
  - Speaker Notes: Add detailed explanations in the notes pane
- 3 No recording is needed.

1. Import packages

List Libraries used

```
# Core ML/DS libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

2. EDA

Dataset used – provide source information or dataset

Step 1: Read the data

```
# Step 1: Load data
df1_combine = pd.read_csv('2025 NFL Combine Dataset.csv')
```

Step 2: Narrow the data

I want to focus only on the running back and wide receiver positions instead of examining all the 7 positions.

```
# Filter the dataset for only the skill positions (RB and WR)
skill_pos = df1_combine[df1_combine['Position'].isin(['RB', 'WR'])].copy()

# Display counts of players at skill positions
print("Skill players: RB={}, WR=".format(len(skill_pos[skill_pos.Position=='RB'])), WR=len(skill_pos[skill_pos.Position=='WR']))
```

```
Skill players: RB=1, WR=40
```

Step 3: Understand the data

```
# Display the first few rows of the dataset
skill_pos.head()
```

	Player	Position	School	Height	Weight	Hand Size	Arm Length	Wingspan	40 Yard Dash	Bench Press	Vertical Jump	Broad Jump	3-Cone Drill	Shuttle	Athleticism Score	Height (Numeric)	Hand (Numeric)	Arm (Numeric)	Wingspan (Numeric)
5	Josh Kelly	WR	Texas Tech	6-0 5/8	186	10 1/2	30 1/8	Nam	4.55	Nam	33	10 1/2	Nam	Nam	5.01000000000000	6	10.5	30.125	Nan
52	Theo Wease Jr.	WR	Missouri	6-2	200	9 5/8	32 1/8	Nam	4.56	Nam	37	9 1/4	Nam	Nam	6.9	6.25	9.625	Nam	0
55	Ricky White III	WR	UNLV	6-1 5/8	184	9 1/4	31 1/4	Nam	4.61	Nam	36	10 1/4	Nam	Nam	6.9	6.0833	9.25	Nam	0
57	Kevin Coleman Jr.	WR	Mississippi State	5-11 3/8	186	9 1/2	29 1/8	Nam	4.59	Nam	36	10 1/2	Nam	Nam	6.92	5.9479166667	9.5	Nan	0
85	Eddie Johnson	RB	Iowa	6-1	224	9 5/8	33	78 1/2	4.57	Nam	Nam	Nam	Nam	Nam	6.94	6	9.625	33	78.5

```
# Display the data types of the array
skill_pos.dtypes
```

```
Name                  object
Position              object
School                object
Height                object
Weight                int64
Hand Size             object
Arm Length            object
Wingspan              object
40 Yard Dash          object
Bench Press           object
Vertical Jump         object
Broad Jump            object
3-Cone Drill          object
Shuttle               object
Athleticism Score     int64
Height (Numeric)      int64
Hand (Numeric)        float64
Arm (Numeric)         float64
Wingspan (Numeric)    object
dtype: object
```

```
# Check which features are missing values to help with feature selection
skill_pos.isna().sum()
```

```
Name                  0
Position              0
School                0
Height                0
Weight                0
Hand Size             0
Arm Length            0
Wingspan              49
40 Yard Dash          0
Bench Press           49
Vertical Jump         23
Broad Jump            23
3-Cone Drill          49
Shuttle               49
Athleticism Score     0
Height (Numeric)      0
Hand (Numeric)        1
Arm (Numeric)         49
Wingspan (Numeric)    0
dtype: int64
```

Step 4: Choose features

```
# These features were chosen because these are the most valuable when analyzing an RB or WR from an NFL scouting perspective
# In addition, we have indeed all RB players, giving RBs more to work with, and 9 of the 10 missing data
# Height (Numeric), Weight, Hand (Numeric), 40 Yard Dash, Wingspan (Numeric)

# Convert each column in the features list to a numeric data type and replace empty strings with NaN
for col in features:
    skill_pos[col] = pd.to_numeric(skill_pos[col], errors='coerce')
```

Step 5: Descriptive Statistics

```
# Examine descriptive statistics of RB and WR combined
skill_pos.describe()
```

	Weight	40 Yard Dash	Bench Press	3-Cone Drill	20 Yard Shuttle	Athleticism Score	Height (Numeric)	Hand (Numeric)
count	40.000000	40.000000	16.000000	13.000000	16.000000	40.000000	40.000000	79.000000
mean	202.867500	4.479625	17.250000	6.859167	4.239625	75.575000	6.008400	9.231152
std	14.786258	0.089517	3.116625	0.125027	0.195090	9.618435	0.195967	0.502392
min	154.000000	4.270000	11.000000	6.650000	4.050000	54.000000	5.666667	7.750000
25%	191.750000	4.405000	15.750000	6.687500	4.100000	68.000000	5.847146	9.000000
50%	204.000000	4.460000	17.000000	6.800000	4.215000	75.000000	6.000000	9.250000
75%	212.250000	4.540000	19.000000	6.977500	4.307500	81.250000	6.148667	9.500000
max	264.000000	4.700000	23.000000	7.100000	4.610000	96.000000	6.479167	10.500000

```
# Examine and display descriptive statistics by skill position for more insight
print(skill_pos.groupby('Position')['Height (Numeric)'].describe())
print(skill_pos.groupby('Position')['Weight'].describe())
```

```
          Height (Numeric)  Hand (Numeric)  40 Yard Dash
Position
RB                  5.8125            8.85          4.55
WR                  6.0040            9.25          4.46
          Height (Numeric)  Hand (Numeric)  40 Yard Dash
Position
RB                  183.0            9.00          4.55
WR                  204.7            9.25          4.46
```

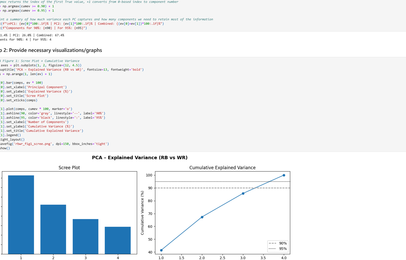
Step 6: Check for missing values

```
# Review key missing values information
skill_pos.isna().sum()[skill_pos.isna().sum() > 0]
missing = skill_pos[features].isna().sum()
missing = missing[missing > 0].sort_values(ascending=False)
print(pd.DataFrame('Missing Count': missing, 'Missing %': pct.))
```

```
Missing Values:
                Missing Count  Missing %
Wingspan                   49       61.3
Bench Press                49       61.3
3-Cone Drill               49       61.3
Shuttle                    49       61.3
Vertical Jump              23       28.8
Broad Jump                 23       28.8
Hand (Numeric)              1        1.2
          Height  Weight  Hand (Numeric)  40 Yard Dash
count  40.000000  40.000              79     40.000000
mean   202.867500 202.875               9      4.479625
std     14.786258  19.977               1      0.089517
min    154.000000 154.000               5      4.270000
25%    191.750000 191.750               9      4.405000
50%    204.000000 204.000               9      4.460000
75%    212.250000 212.250               9      4.540000
max    264.000000 264.000               9      4.700000
Position
RB      183.0      183.0            9.00       4.55
WR      204.0      204.0            9.25       4.46
          Height  Weight  Hand (Numeric)  40 Yard Dash
Position
RB      183.0      183.0            9.00       4.55
WR      204.0      204.0            9.25       4.46
```

Step 7: Plot the respective graphs

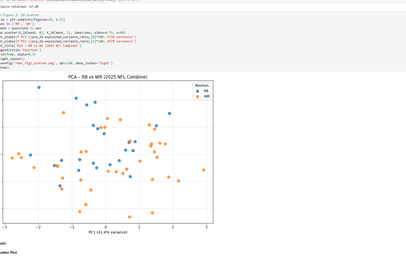
```
# Step 7: Boxplots
fig, axes = plt.subplots(1, 4, figsize=(14, 5))
fig.suptitle('Feature Distributions by Position (RB vs WR)', fontsize=15, fontweight='bold')

for i, feat in enumerate(features):
    sns.boxplot(data=skill_pos, x='Position', y=feat, ax=axes[i])
    axes[i].set_title(feat, fontsize=11, fontweight='bold')
    axes[i].grid(axis='y')
    axes[0].set_title('Weight (lbs)', fontsize=11, fontweight='bold')
    axes[1].set_title('Height (in)', fontsize=11, fontweight='bold')
    axes[2].set_title('Hand Size (in)', fontsize=11, fontweight='bold')
    axes[3].set_title('40 Yard Dash (sec)', fontsize=11, fontweight='bold')

plt.tight_layout()
plt.savefig('nfl_fig1_boxplots.png', dpi=150, bbox_inches='tight')
plt.show()
```



Analysis:

- RBs are heavier (mean of 208.8 lbs vs. 196.8 lbs) and shorter (5.8 ft vs. 6.00 ft) than WRs. This makes sense because historically and generally, RBs require a lower center of gravity and contact tolerance.
- RBs have slightly larger hands on average (9.17 in vs 9.25 in). This is consistent with their role because larger hands help them catch the ball better.
- Both positions can nearly identical 40 Yard Dash times (4.46 vs 4.46 sec). In conclusion, speed is a universal baseline at these skill positions regardless of size.

```
# Step 8 Figure 2: Correlation Heatmap
fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(skill_pos_clean[features].corr(), annot=True, fmt='.2f', cmap='RdBu_r', center=0, ax=ax)
ax.set_title('Feature Correlation Matrix - RB & WR', fontsize=15, fontweight='bold')
plt.tight_layout()
plt.savefig('nfl_fig2_correlation.png', dpi=150, bbox_inches='tight')
plt.show()
```



Analysis:

- Feature correlations are from 0.10-0.40. This is moderate and means that each it including measurements are independent of one another and contributes something to the analysis.
- The strongest correlation is 0.40 between Height and Hand Size. This makes sense because taller players tend to have longer fingers.
- The near-zero correlation of -0.02 between Hand Size and 40 Yard Dash make sense and confirms speed and hand size are completely independent of one another.

3. PCA

Step 1: Feature Scaling

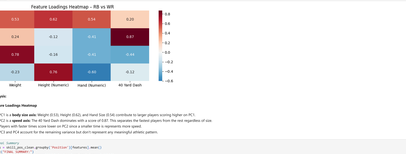
```
# Weight (lbs) and 40 Yard Dash (sec) are on very different scales, so StandardScaler gives each feature mean=0, std=1 so that none dominates PCA

# Creates an instance of the StandardScaler and name it scaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
# Full PCA - Explained Variance

# Fit PCA using all 5 components so we can see the full variance picture
pca_full = PCA()
X_pca_full = pca_full.fit_transform(X_scaled)

# Calculate cumulative ratio, gives the % of variance each PC captures
cum_var = np.cumsum(pca_full.explained_variance_ratio_)

# Cumulative sum adds them up progressively
cum_var = np.cumsum(pca_full.explained_variance_ratio_)

# Find the minimum number of components needed to hit the 90% and 95% thresholds
# np.argmax returns the index of the first True value, +1 converts from 0-based index to component number
n90 = np.argmax(cum_var >= 0.90) + 1
n95 = np.argmax(cum_var >= 0.95) + 1

# Print a summary of how many components we need to retain most of the information
print("Total features: {}".format(len(features)), f"Components for 90% variance: {n90}")
print("Components for 95% variance: {n95}")
```

```
Total features: 5 Components for 90% variance: 4
Components for 95% variance: 5
```

Step 2: Provide necessary visualizations/graphs

```
# Step 2 Figure 3: Scree Plot + Cumulative Variance
fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
axes[0].bar(range(1, len(pca_full.explained_variance_ratio_) + 1), pca_full.explained_variance_ratio_, alpha=0.7, color='steelblue', edgecolor='black')
axes[0].plot(range(1, len(pca_full.explained_variance_ratio_) + 1), pca_full.explained_variance_ratio_, 'o-', color='red', linewidth=2, markersize=8)
axes[0].set_xlabel('Principal Component')
axes[0].set_ylabel('Explained Variance (%)')
axes[0].set_title('Scree Plot')
axes[0].set_xticks(range(1, len(pca_full.explained_variance_ratio_) + 1))
axes[0].grid(axis='y')
axes[1].plot(range(1, len(cum_var) + 1), cum_var, 'o-', color='red', linewidth=2, markersize=8)
axes[1].axhline(y=0.90, color='green', linestyle='--', label='90%')
axes[1].axhline(y=0.95, color='blue', linestyle='--', label='95%')
axes[1].set_xlabel('Number of Components')
axes[1].set_ylabel('Cumulative Explained Variance (%)')
axes[1].set_title('Cumulative Explained Variance')
axes[1].legend()
axes[1].grid(axis='y')
plt.tight_layout()
plt.savefig('nfl_fig3_scree.png', dpi=150, bbox_inches='tight')
plt.show()
```



Analysis:

Scree Plot & Cumulative Variance

- PC1 captures 41.4% and PC2 captures 26.0% of the variance. These 2 dimensions capture 67.4% of the total information.
- All 5 components are needed to capture 90% variance, with PC1+PC2+PC3+PC4 contributing much smaller amounts than PC1 and PC2.
- No single feature dominates and each scooping measurement contributes something unique as indicated by the gradual scree slope.

Step 3: Reduce to 2 Components

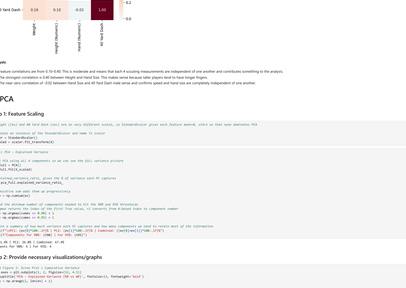
Dimensionality reduction

We reduce to 2 dimensions because PC1 and PC2 already capture the dominant structure in our data. Everything after that is diminishing returns and not that significant to our purpose.

```
pca_2d = PCA(n_components=2)
X_2d = pca_2d.fit_transform(X_scaled)
print("2D variance retained: {:.1f}".format(pca_2d.explained_variance_ratio_[0]*100, 1f}")
```

```
2D variance retained: 67.4%
```

```
# Step 3 Figure 4: 2D Scatter
fig, ax = plt.subplots(figsize=(9, 6.5))
for pos in ['RB', 'WR']:
    mask = positions == pos
    ax.scatter(X_2d[mask, 0], X_2d[mask, 1], label=pos, alpha=0.70, s=80)
ax.set_xlabel('PC1 (41.4% explained variance_ratio_[0]*100: .1f}% variance)')
ax.set_ylabel('PC2 (26.0% explained variance_ratio_[1]*100: .1f}% variance)')
ax.set_title('PCA - RB vs WR (2025 NFL Combine)', fontsize=15, fontweight='bold')
ax.legend(title='Position')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('nfl_fig4_scatter.png', dpi=150, bbox_inches='tight')
plt.show()
```



Analysis:

2D Scatter Plot

- PC1 represents body SIZE (Weight, Height, and Hand Size)
- PC2 represents SPEED (40 Yard Dash)
- RBs and WRs show partial but meaningful separation along PC1, with RBs clustering at higher values. This is indicative of their heavier, compact frames.
- RBs spread more broadly. This is indicative of the variety of body types at this position because usually taller WRs are boundary receivers, where shorter WRs are slot receivers.
- The overlap in the middle can represent hybrid players where RB either position physically. This is consistent with the modern NFL trend toward positionless skill players in RBs that have run plays/kick returners or pass catching RBs

```
# Step 4 Figure 5: Loadings Heatmap
# Shows what each PC physically represents and which features drive it
fig, ax = plt.subplots(figsize=(8, 4))
    pca_full.components_,
    index=['PC1', 'PC2', 'PC3', 'PC4', 'PC5'],
    columns=features
fig, ax = plt.subplots(figsize=(8, 3.5))
sns.heatmap(loadings_df, annot=True, fmt='.2f', cmap='RdBu_r', center=0, ax=ax)
ax.set_title('Feature Loadings Heatmap - RB vs WR')
plt.tight_layout()
plt.savefig('nfl_fig5_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
```


Analysis:

Feature Loadings Heatmap

- PC1 is a **body size axis**: Weight (0.53), Height (0.62), and Hand Size (0.54) contribute to larger players scoring higher on PC1.
- PC2 is a **speed axis**: The 40 Yard Dash dominates with a score of 0.60. This separates the fastest players from the rest regardless of size.
- Players with faster times score lower on PC2 since a smaller time is represents more speed.
- PC3 and PC4 account for the remaining variance but don't represent any meaningful athletic pattern.

```
# Print Summary
means = skill_pos_clean.groupby('Position')[features].mean()
print("TOTAL SUMMARY")
print("Total Players: {}".format(len(skill_pos_clean)))
print("PC1 Explained: {:.1f}%".format(pca_full.explained_variance_ratio_[0]*100))
print("PC2 Explained: {:.1f}%".format(pca_full.explained_variance_ratio_[1]*100))
print("Total Variance: {:.1f}%".format(pca_full.explained_variance_ratio_[:2].sum()*100))
print("RB avg weight: {means.loc['RB', 'Weight']:.1f} lbs | WR: {means.loc['WR', 'Weight']:.1f} lbs")
print("RB avg height: {means.loc['RB', 'Height (Numeric)']:.1f} in | WR: {means.loc['WR', 'Height (Numeric)']:.1f} in")
print("RB avg hand: {means.loc['RB', 'Hand (Numeric)']:.1f} in | WR: {means.loc['WR', 'Hand (Numeric)']:.1f} in")
print("RB avg 40 yard: {means.loc['RB', '40 Yard Dash']:.2f} sec | WR: {means.loc['WR', '40 Yard Dash']:.2f} sec")

FINAL SUMMARY:
Total Players: 40
PC1 (35.4%): Size/Frame - Weight, Height, Hand Size
PC2 (24.8%): Speed - 40 Yard Dash
Total Variance: 60.2%
RB avg weight: 208.8 lbs | WR: 196.8 lbs
RB avg height: 5.8 in | WR: 6.00 in
RB avg hand: 9.17 in | WR: 9.25 in
RB avg 40 yard: 4.46 sec | WR: 4.46 sec
```

Concluding Statement:

The PCA results show that the physical skills combine measurements we looked at can be collapsed into 2 primary athletic dimensions SIZE and SPEED. Of course there are other factors that make a good RB and WR such as route running ability, catching ability, etc. but for the purpose of this analysis, these are the two major factors that help determine a players athletic ability for these skill positions.